SHAP-Coded Insights: Interpretable Structural Similarity of Programs with GraphCodeBERT

Amal S
Dept. of Computer Science and Applications
Amrita School of Computing
Amrita Vishwa Vidyapeetham, Amritapuri
amalsreekumar94@gmail.com

Niranjan A. R
Dept. of Computer Science and Applications
Amrita School of Computing
Amrita Vishwa Vidyapeetham, Amritapuri
arniranjan477@gmail.com

M. G. Thushara
Dept. of Computer Science and Applications
Amrita School of Computing
Amrita Vishwa Vidyapeetham, Amritapuri
thusharamg@am.amrita.edu

***Abstract*—Assessing the similarity between source code submissions plays an essential role in programming education. While traditional tools detect copied code, they often fall short when students implement logically equivalent solutions using different structures. This study introduces a code similarity analysis method that integrates semantic embeddings from GraphCodeBERT, structural comparisons through abstract syntax trees (ASTs), and lexical token overlap. To promote transparency in evaluation, SHAP-based attribution is employed to identify which elements contribute most to the similarity score. The system was tested on a dataset comprising C and Java student submissions and corresponding instructor solutions. It produced similarity scores ranging from 0.77 to 1.00 and generated targeted feedback based on structural and semantic features. These findings indicate the model's potential for providing both accurate assessment and actionable feedback in programming courses.**

***Index Terms*—Code Similarity, Programming Education, GraphCodeBERT, Abstract Syntax Tree, Explainable AI**

I. Introduction

Code similarity analysis is an essential task in software engineering, with applications in automated grading, plagiarism detection, and code maintenance [1], [2]. In academic settings, it supports instructors in assessing student work fairly and helps uphold academic integrity by detecting copied or reused code [3], [4].

Traditional similarity tools like MOSS and JPlag rely mainly on token-based techniques. While effective for identifying near-exact matches (Type-1 and Type-2 clones), they often fail to recognize logically equivalent code written in different styles. For instance, two functionally similar programs—one using iteration and the other recursion—may receive low similarity scores due to structural differences.

This limitation is particularly problematic in programming education, where students may choose different implementation approaches that are both correct. More importantly, these tools do not explain how similarity scores are derived, making it difficult for learners to understand their mistakes or improve their code. This lack of transparency reduces their effectiveness as educational aids [5].

Recent advances in pretrained models based on the Transformer architecture have improved the semantic understanding of source code. Models such as BERT [6] and CodeBERT [7], [8] capture contextual information, but are still limited to token-level analysis and do not account for structural relationships. Moreover, these models operate as black boxes, offering little insight into how similarity judgments are made.

To overcome these issues, we propose a model that combines semantic, structural, and lexical features with interpretability to assess code similarity. Our method integrates the following components:

- **GraphCodeBERT** [9], [10]: Generates embeddings that capture both control and data flow, enabling semantic comparison.
- **AST-based similarity** [11]: Compares structural elements of programs to detect logical alignment.
- **Token overlap** [12]: Measures lexical similarity based on common tokens.
- **SHAP explanations** [13], [14]: Attribute feature importance to explain how similarity scores are influenced.

By combining these components, the model produces both a similarity score and targeted feedback. This helps students identify where their code differs from the expected solution and supports instructors in evaluating varied submissions. In educational contexts, such interpretable feedback is more beneficial than a score alone, as it encourages self-reflection and continuous improvement.

The paper is organized as follows: Section II reviews existing studies and key technologies. Section III introduces the methodology developed. Section IV examines the experimental results based on actual submissions of students . Section V provides a detailed discussion of the findings.Section VI concludes the study and suggests possible recommendations for future exploration.

II. Related Work and Background

This section outlines the technologies and research that form the basis of our approach. It focuses on four areas: code structure representations, pretrained models for code understanding, explainability in AI, and baseline methods for textual comparison. Together, these elements support a system that not only detects similarity [15], but also provides constructive feedback for learners [1].

A. Structural Representations of Code

Abstract Syntax Trees (ASTs) are commonly used to capture the structural layout of code beyond its surface syntax [11], [16]. They represent key programming constructs—such as loops, conditionals, and function definitions—in a tree-like hierarchy [17] that supports deeper comparisons than text alone.

To generate ASTs, we use language-specific parsers:

- `pycparser` [18]: A parser for C that builds complete ASTs.
- `javalang` [19]: A Python-based parser for Java source code.

These ASTs allow node-level matching and structural difference detection, which is especially useful when analyzing varied implementations of the same logic.

B. Pretrained Models for Code Understanding

Recent advances in pretrained models have improved the ability to understand code across languages and styles. These models learn both syntax and semantics by training on large repositories of source code.

- **BERT** [6]: A language model originally trained on text, useful for capturing contextual token relationships.
- **CodeBERT** [7], [8]: A variant of BERT adapted for code and documentation pairs. It works well for tasks like retrieval and summarization, but does not explicitly model structural relationships.
- **GraphCodeBERT** [9], [20]: Enhances CodeBERT by integrating data-flow edges, helping the model learn control and dependency structures. This makes it better suited for comparing code that may differ in form but not function. GraphCodeBERT plays a central role in our system.

C. Explainability and SHAP

In educational settings, it is not enough to provide a similarity score; learners benefit from understanding why two pieces of code are similar or different. Explainable AI helps bridge this gap.

SHAP (SHapley Additive exPlanations) [13], [14] is a framework that assigns importance scores to features by estimating their individual contribution to a model's output. In our case, it helps identify which parts of the student code had the most influence on the similarity score.

Key benefits of SHAP in this context include:

- Identifying which nodes or tokens influenced the score.
- Highlighting the structural or lexical elements responsible for similarity or mismatch.
- Enabling clearer feedback by linking explanation to specific code regions.

This interpretability supports both learning and instruction, allowing students to see what needs improvement and helping instructors understand student approaches at a structural level.

D. Text-Based Comparison Baseline

We also incorporate `DiffLib` [12], a Python module that compares text sequences line by line. It serves as a simple baseline for comparison.

- **Language-agnostic:** Works with any text, regardless of syntax.
- **Efficient:** Suitable for quick diffs and visual inspection.

However, DiffLib does not consider the semantics or structure of code, and cannot distinguish between different implementations that achieve the same result. Therefore, it is primarily used to benchmark the improvements offered by structure-aware and interpretable models like GraphCodeBERT.

Although prior work has focused on code similarity and clone detection, few approaches have been adapted specifically for educational use. Our work fills this gap by combining code understanding and explainability to support formative assessment and guided feedback in programming courses.

III. Methodology

This section describes the interpretable code similarity pipeline used to analyze student submissions. The system combines semantic embeddings, structural comparison, and feature attribution to provide both similarity scores and feedback.

An overview of the proposed pipeline is illustrated in Fig. 1 The process begins with student code submissions that are passed through a pre-processing module. The preprocessed code is then tokenized and embedded using GraphCodeBERT and parsed into Abstract Syntax Trees (ASTs). Similarity scores are computed based on both embeddings and AST structures. These scores are fed into a surrogate model for SHAP-based explanation, which finally leads to tailored feedback generation through a unified diff visualization.

A. Dataset and Preprocessing

The dataset consists of C and Java programming assignments, including instructor solutions and student submissions. Preprocessing includes:

- **Normalization:** Comments and excess whitespace are removed to produce $C_{\text{norm}} = f_{\text{norm}}(C_{\text{raw}})$.
- **Language Detection:** Heuristic detection function $L = h(C_{\text{norm}})$, where $L \in \{\text{Java}, \text{C}\}$.
- **C-specific Handling:** For C programs, header macros are preprocessed using $C_{\text{norm}} = P(C_{\text{norm}})$.
- **Tokenization and Parsing:** Code is tokenized using GraphCodeBERT's tokenizer and parsed into ASTs.

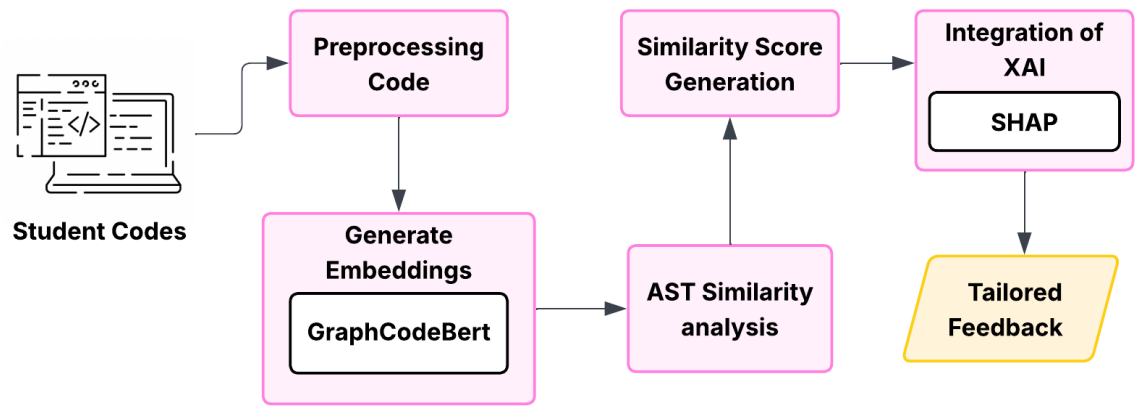

Fig. 1: Overview of the code similarity analysis pipeline.

B. GraphCodeBERT for Code Embedding:

GraphCodeBERT, a pre-trained transformer model trained on a large code set, is used to generate vector representations (embeddings) of source code in order to compare similarities. The steps include:

- The input code is first tokenized into a sequence : $T = \{t_1, t_2, ..., t_n\}$, where each T represents an indivisual token.
- The model then generates contextual embeddings: $H = [h_1, h_2, ..., h_n]$, with $h_i \in \mathbb{R}^d$ denoting the hidden state for the token t_i
- The code embedding $E \in \mathbb{R}^d$ is computed as the mean of all token embeddings:

$$E = \frac{1}{n}\sum_{i=1}^{n} h_i \tag{1}$$

  where n indicates the total number of tokens.
- To compare two code snippets with embeddings E_1 and E_2, cosine similarity is computed as:

$$S_{GCB} = \frac{E_1 \cdot E_2}{\|E_1\|\|E_2\|} \tag{2}$$

This approach enables the model to capture both syntactic (e.g., loops, conditionals) and semantic (e.g., data flow, variable usage) relationships through its pre-trained weights. By not fine-tuning the model, we maintain efficiency and generalizability, though it may lack precision for domain-specific tasks. However, cosine similarity is a good measure for comparing logical and structural similarity between code snippets.

C. AST Similarity Analysis:

This step measures structural similarity using Abstract Syntax Trees (ASTs). The steps include:

- Parsing preprocessed code into ASTs: $A_1 = \text{parse}(C_{norm1}, L)$, $A_2 = \text{parse}(C_{norm2}, L)$.
- Extracting node sets: $N_1 = \{n_{1i}\}$ from A_1, $N_2 = \{n_{2j}\}$ from A_2.
- Finding common nodes: $N_{common} = N_1 \cap N_2$.
- Computing similarity:

$$S_{AST} = \frac{|N_{common}|}{\max(|N_1|, |N_2|)} \tag{3}$$

Where N_1, N_2 are node sets extracted from the ASTs,

D. SHAP-Based Explainability

SHAP interprets similarity scores using a surrogate model (e.g., random forest) since GraphCodeBERT is not SHAP-compatible. The workflow is:

- Train a surrogate model

$$M(F) = S_c \text{ on features } F = [S_{GCB}, S_{AST}, S_{token}, l_1, l_2]. \tag{4}$$

- Compute SHAP values:

$$SHAP_i = \phi_i(M, F) \tag{5}$$

- Map $SHAP_i$ to dissimilar AST nodes:

$$n_{1i} \in N_1 \setminus N_{common}, \quad n_{2j} \in N_2 \setminus N_{common} \tag{6}$$

- Score dissimilarity:

$$D_i = |SHAP_i| \tag{7}$$

- Generate unified diff:

$$\text{Diff} = \text{unified_diff}(C_{raw1}, C_{raw2}) \tag{8}$$

- Generate final feedback:

$$F_{out} = f_{format}(SHAP, \text{Diff}) \tag{9}$$

Where:

- M is the surrogate model,
- F is the input feature vector,
- ϕ_i is the SHAP value for feature i,
- D_i is the dissimilarity contribution of feature i,
- Diff shows line-level differences between student and correct code,

Since GraphCodeBERT is not directly SHAP-compatible, a surrogate random forest is used to approximate the total

similarity score. The SHAP values quantify the contribution of embedding similarity, AST similarity, and token overlap contributions into the overall score, identifying key differences. Dissimilar nodes in the ASTs are evaluated on the basis of their SHAP values, identifying structural deviations. Feedback combines these results with an overall diff, offering actionable recommendations for educational improvement.

E. Scoring Mechanism

The overall similarity score is computed as a weighted sum:

$$S_c = 0.5 \cdot S_{GCB} + 0.35 \cdot S_{AST} + 0.15 \cdot S_{token} \tag{10}$$

Where:

- S_{GCB} is the semantic similarity from embeddings,
- S_{AST} is the structural similarity from AST comparison,
- S_{token} is the lexical similarity based on token overlap.

Similarity categories are assigned as follows:

$$\text{Category}(S_c) = \begin{cases} \text{Excellent}, & \text{if } S_c > 0.98 \\ \text{Great}, & \text{if } 0.90 < S_c \leq 0.98 \\ \text{Partial}, & \text{if } 0.65 < S_c \leq 0.90 \\ \text{Significant}, & \text{if } S_c \leq 0.65 \end{cases} \tag{11}$$

where:

- S_c — Combined similarity score
- **Excellent**: Nearly identical code
- **Great**: Minor structural or logical differences
- **Partial**: Functionally related, structurally distinct
- **Significant**: Major structural or logical divergence

These categories help to map scores to feedback descriptions and guide learners in improving their code effectively.

F. Feedback and Visualization

The final output includes:

- Categorized similarity label
- Highlighted AST segments with high SHAP values
- Token overlap details
- Unified diff annotations

This combination offers both quantitative and qualitative insights into code differences, aiding formative feedback in educational settings.

IV. Results

To evaluate the effectiveness of our code similarity and feedback System, we have executed it on a set of real student code submissions in C and Java. All the submissions have been compared with a correct program using three levels of analysis: GraphCodeBERT embeddings, token overlap, and Abstract Syntax Tree (AST) similarity.

The results from the experiments are summarized in Table I & II. The similarity scores (in Table I) observed across the fifteen student-reference code pairs range from 0.77 to 1.00. Among these, Pair 5 and Pair 12 achieved a perfect score of 1.00, indicating complete alignment with the correct solutions. Pairs such as 1, 4, 6, and 7, which scored between 0.90 and 0.97, were accompanied by encouraging feedback (refer Table II), suggesting that only minor improvements were needed to fully match the expected outputs. In contrast, Pairs 13, 14, and 15, which had lower similarity scores between 0.77 and 0.80, showed more noticeable differences in structure and logic. The feedback for these cases pointed to the need for structural refinements to bring the implementations closer to the intended solutions.

Table I & II summarizes version of the original output for each pair. The complete output contains more information than the similarity score and feedback, including a difference breakdown between the student and correct codes, as well as contributing factors to the score. For example, in the case of the pair model gives a similarity score of 0.91 (Refer student code `CircleAreaSimple.java` in Listing 1 vs. correct code `CircleAreaFormula.java` in Listing 2).

TABLE I: Code Similarity Scores for Student–Correct Code Pairs

Pair	Student Code	Correct Code	Score
1	CircleAreaSimple.java	CircleAreaFormula.java	0.91
2	ReverseLoop.java	ReverseRecursive.java	0.84
3	SquareRecursive.java	SquareLoop.java	0.84
4	ArmstrongLoop.java	ArmstrongRecursive.java	0.91
5	ArmstrongLoop.java	ArmstrongLoop.java	1.00
6	ReverseRecursive.java	SquareRecursive.java	0.90
7	ArmstrongLoop.c	ArmstrongRecursive.c	0.97
8	TempConvertDirect.c	TempConvertTable.c	0.79
9	FactorialIterative.c	FactorialFormula.c	0.84
10	PalindromeLoop.c	PalindromeRecursive.c	0.87
11	SquareLoop.c	SquareRecursive.c	0.85
12	PalindromeRecursive.c	PalindromeRecursive.c	1.00
13	SquareRecursive.c	TempConvertTable.c	0.78
14	CircleAreaFormula.java	ArmstrongLoop.java	0.77
15	PalindromeRecursive.c	SquareLoop.c	0.80

TABLE II: Feedback Based on Similarity Scores (refer Table I)

Pair	Feedback
1	Very close to the correct code with minor logic differences.
2	Works correctly, but structure differs noticeably.
3	Works correctly, but structure differs noticeably.
4	Very close to the correct code with minor logic differences.
5	Excellent match with the correct code.
6	Very close to the correct code with minor logic differences.
7	Very close to the correct code with minor logic differences.
8	Functional, but structurally different from the expected version.
9	Works correctly, but structure differs noticeably.
10	Works correctly, but structure differs noticeably.
11	Works correctly, but structure differs noticeably.
12	Excellent match with the correct code.
13	Functional, but structurally different from the expected version.
14	Functional, but structurally different from the expected version.
15	Works correctly, but structure differs noticeably.

Listing 1: CircleAreaSimple.java

```
import java.util.Scanner;

public class SimpleCircleArea {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double radius = scanner.nextDouble();
        double area = Math.PI * Math.pow(radius, 2);
        System.out.printf("The area is: %.2f\n", area);
        scanner.close();
    }
}
```

Listing 2: CircleAreaFormula.java

```
import java.util.Scanner;
public class CircleAreaWithFormula {
    public static double computeCircleArea(double radius) {
        return Math.PI * Math.pow(radius, 2);
    }
public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);
        double radius = inputScanner.nextDouble();
        double area = computeCircleArea(radius);
        System.out.println("Calculated Area: " + String.
            format("%.2f", area));
        inputScanner.close();
    }
}
```

Listing 3 presents the unified difference between the student's code and the instructor's reference implementation. Lines prefixed with '-' indicate code segments removed or replaced, while those with '+' indicate additions from the correct version. This output enables learners to visually track structural and functional differences, such as the use of constants or modular code design.

Listing 3: Unified difference in output between student code and correct reference

```
--- Student
+++ Correct
@@ -1,11 +1,14 @@
-public class CircleAreaSimple {
+public class CircleAreaFormula {
+    public static float calculateArea(float r) {
+        return (float) (Math.PI * r * r);
+    }

-    float area = 3.14f * radius * radius;
-    System.out.printf("%.2f\n", area);
+    System.out.printf("%.2f\n", calculateArea(
    ↪ radius));
```

The student code uses a class **CircleAreaSimple** and calculates the area inline with a constant value of $3.14f$ for π, while the correct code calls a method **calculateArea** with Math.PI for precision.

Sample SHAP Explanation

Here's why these codes are similar or different:

- **GraphCodeBERT similarity:** Increases similarity by 0.0192
- **AST similarity:** Decreases similarity by 0.0110
- **Token overlap:** Decreases similarity by 0.0252
- **Tokens in student code:** Increases similarity by 0.0020
- **Tokens in correct code:** Decreases similarity by 0.0230
- **Lines in student code:** Increases similarity by 0.0124

This breakdown indicates that the overall purpose and common elements increased the score slightly, while differences in organization and the number of lines or components reduced it, leading to the feedback advising structural adjustments to align with the expected solution.

The results show that the code similarity and feedback tool is a useful tool for assessing students' submissions. The score range, 0.77 to 1.00, shows that the tool can distinguish between submissions that closely match a correct answer and those with significant differences. This flexibility in C and Java submissions indicates that it may be useful in many programming situations. The system is so strong because of its elaborate feedback, which offers customized advice to enable students to correct their work, whether by adjusting structure or addressing logical gaps. This individualized support makes it that much more of an educational tool.

From an instructional perspective, these results enable targeted interventions where students show recurring structural mismatches. For students, the feedback encourages reflective learning and iterative improvement.

V. Discussion

The evaluation results demonstrate that the proposed model is capable of differentiating between submissions that are structurally similar and those that deviate significantly from the reference code. For instance, submissions like Pair 5 and Pair 12, which closely matched the expected output, received perfect scores, while others with alternate logic or structure, such as Pairs 13 and 14, obtained lower scores.

One notable observation is that recursive and iterative implementations of the same problem were often rated with moderately high similarity (e.g., Pairs 2 and 3). This suggests that the combination of AST structure and semantic embeddings is effective in recognizing underlying intent, despite differences in control flow. However, token-level overlap had limited influence when structural and semantic components diverged sharply.

The SHAP explanations provided insights into how specific features contributed to the final score. In several cases, large dissimilar SHAP values corresponded with method reorganization or use of hard-coded constants, allowing for precise

feedback. These explanations are especially valuable in educational settings, where learners benefit from understanding both their errors and how to improve.

Nonetheless, the model showed limitations in dealing with malformed code or syntax inconsistencies, which affected parser performance. Additionally, some pairs with different functional goals but lexical similarities (e.g., shared variable names or print statements) resulted in slightly inflated similarity scores. Future enhancements may include filtering irrelevant tokens or adjusting the scoring weights dynamically based on code structure.

VI. Conclusion and Future Work

This study proposed an approach for code similarity analysis aimed at supporting programming education. By integrating semantic embeddings, structural comparison through ASTs, and lexical overlap, the system produces a combined similarity score along with interpretable feedback. The incorporation of SHAP-based attribution allows educators and learners to understand which components of the code most influenced the similarity assessment.

Evaluated on real-world student submissions in C and Java, the system successfully distinguished between highly similar, partially similar, and structurally distinct code. Feedback aligned closely with instructor expectations, indicating its value in formative assessment. Despite some challenges in handling poorly formatted code and borderline cases, the model proved effective in recognizing logical equivalence and offering clear guidance for improvement.

Future Work

- Improve feedback by incorporating program behavior and context.
- Extend support to additional languages such as Python and C++.
- Use adaptive learning techniques to refine feedback based on usage.
- Fine-tune similarity weights to better handle varied coding practices.

References

[1] R. Parvathy, M. G. Thushara, and J. M. Kannimoola, "Automated code assessment and feedback: A comprehensive model for improved programming education," *IEEE Access*, vol. 13, pp. 56 642–56 658, 2025.

[2] M. Hu and Y. Zhang, "The python/c api: evolution, usage statistics, and bug patterns," in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2020, pp. 532–536.

[3] M. V. D. Naik and S. Jayaraman, "Cross-language code mapping with transformer encoder-decoder model," in *Intelligent Informatics*, S. K. Pal, S. M. Thampi, and A. Abraham, Eds. Singapore: Springer Nature Singapore, 2025, pp. 439–458.

[4] S. Thangavelu, V. Rao, C. Shyamala, and C. S. Velayutham, "Introductory programming using non-textual modalities-an empirical study on skill assessment using rainfall problem," in *2019 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, 2019, pp. 867–871.

[5] M. D. Naik and S. Jayaraman, "Cross-language code mapping," in *Intelligent Informatics: Proceedings of Eighth International Symposium on Intelligent Informatics (ISI 2023)*, vol. 389. Springer Nature, 2025, p. 439.

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[7] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. [Online]. Available: https://arxiv.org/abs/2002.08155

[8] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, "CodeBERTScore: Evaluating code generation with pretrained models of code," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, December 2023, pp. 13 921–13 937. [Online]. Available: https://aclanthology.org/2023.emnlp-main.859/

[9] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, J. Yin, M. Zhou, and N. Sundaresan, "Graphcodebert: Pre-training code representations with data flow," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://arxiv.org/abs/2009.08366

[10] J. Martinez-Gil, "Improving source code similarity detection through graphcodebert and integration of additional features," *arXiv preprint arXiv:2408.08903*, 2024. [Online]. Available: https://arxiv.org/abs/2408.08903

[11] W. Sun, C. Fang, Y. Miao, Y. You, M. Yuan, Y. Chen, Q. Zhang, A. Guo, X. Chen, Y. Liu, and Z. Chen, "Abstract syntax tree for programming language understanding and representation: How far are we?" *arXiv preprint arXiv:2312.00413*, 2023. [Online]. Available: https://arxiv.org/abs/2312.00413

[12] P. S. Foundation, *difflib — Helpers for Computing Deltas*, Python Documentation, https://docs.python.org/3/library/difflib.html, 2024, accessed: 2025-04-09.

[13] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1705.07874

[14] E. Mosca, F. Szigeti, S. Tragianni, D. Gallagher, and G. Groh, "Shap-based explanation methods: A review for nlp interpretability," in *Proceedings of the 29th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, October 2022, pp. 4593–4603. [Online]. Available: https://aclanthology.org/2022.coling-1.406/

[15] L. Mohith, M. Manikandan, and M. Thushara, "An exploration of code similarity and code replication in computational programs," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2024, pp. 1–5.

[16] O. Büyük and A. Nizam, "Deep learning with class-level abstract syntax tree and code histories for detecting code modification requirements," *Journal of Systems and Software*, vol. 206, p. 111377, 2023. [Online]. Available: https://acikerisim.fsm.edu.tr/xmlui/handle/11352/4656

[17] S. Rajesh, V. V. Rao, and M. Thushara, "Comprehensive investigation of code assessment tools in programming courses," in *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, 2024, pp. 1–6.

[18] E. Bendersky, "pycparser: A complete parser of the c language in pure python," https://github.com/eliben/pycparser, 2008, accessed: 2025-04-09.

[19] C2NES, "javalang: Pure python library for parsing java source code," https://github.com/c2nes/javalang, 2014, accessed: 2025-04-09.

[20] J. Martinez-Gil, "Augmenting the interpretability of graphcodebert for code similarity tasks," *arXiv preprint arXiv:2410.05275*, 2024. [Online]. Available: https://arxiv.org/abs/2410.05275